

STARBASE MATERIAL SELECTION SYSTEM

PROJECT REPORT

Submitted by

SUDHEESH P
Roll No: 64
University ID: LTVE24EL071

BABITHOSH EB
Roll No: 18
University ID: LTVE24EL069

MINNA ANTONY
Roll No: 47
University ID: TVE24EL044

SEBIN XAVIER
Roll No: 60
University ID: TVE24EL056

B.Tech Electrical and Computer Engineering
Department of Electrical Engineering
College of Engineering Trivandrum

Academic Year 2026–2027

CERTIFICATE OF COMPLETION

This is to certify that Sudheesh P (Roll No. 64), Babithosh EB (Roll No. 18), Minna Antony (Roll No. 47), and Sebin Xavier (Roll No. 60), B.Tech Electrical and Computer Engineering students of College of Engineering Trivandrum, have completed the work presented in the report titled “Starbase Material Selection System” at College of Engineering Trivandrum, as part of the Database Management System (PBEOT504) Project-Based Learning (PBL) curriculum. This is a bonafide record of their original work carried out under our supervision and guidance.

Prof. Muhammed Raseem S.
Department of Computer Science & Engineering
College of Engineering Trivandrum
Signature: _____ Date: _____

Prof. Julie T.S.
Department of Electrical Engineering
College of Engineering Trivandrum
Signature: _____ Date: _____

DECLARATION

We hereby declare that the project report entitled “Starbase Material Selection System” is a record of the work carried out by our group as part of the Project-Based Learning (PBL) course Database Management System (PBEOT504) under the guidance of Prof. Muhammed Raseem S. and Prof. Julie T.S.

We further declare that this report has been prepared as part of the academic requirements of the B.Tech degree programme and that the work presented herein has not been submitted, either in whole or in part, to any other institution for the award of any degree or diploma.

We have duly acknowledged all sources of information used in the preparation of this report.

TRIVANDRUM
26-10-2026

Sudheesh P (Roll No. 64, TVE ID: LTVE24EL071) Signature: _____

Babithosh EB (Roll No. 18, TVE ID: LTVE24EL069) Signature: _____

Minna Antony (Roll No. 47, TVE ID: TVE24EL044) Signature: _____

Sebin Xavier (Roll No. 60, TVE ID: TVE24EL056) Signature: _____

ABSTRACT

Selecting an optimal engineering material for a mechanical or aerospace component requires balancing conflicting design objectives, including mass minimization, structural yield strength, stiffness, thermal endurance, and commercial availability. While vast computational and empirical materials data exists, it remains fragmented across incompatible repositories, proprietary silos, and non-standardized datasheets. Starbase is an open, relational database system and engineering selection workspace designed to unify multi-source scientific materials data into a single, fully normalized catalog.

The database schema implements a third-normal-form (3NF) property-dictionary architecture that reconciles 6,265,691 material records across 14 authoritative open repositories, including NIST JARVIS-DFT, Materials Project (core and GNoME), AFLOW, NOMAD, Alexandria, NASA TPSX, spacecraft outgassing archives, high-entropy alloy datasets, and certified stainless-steel supplier catalogs. Complete source provenance (origin repository, upstream version, license terms, digital checksum, and evidence classification) is captured for every record. The database footprint comprises 18 core application schema tables (17 active application tables and one legacy review table), expanding to 20 tables in the cloud deployment with two dedicated migration and audit support tables.

The selection engine applies a hybrid evaluation strategy: mandatory engineering constraints are evaluated directly in SQL via composite-indexed criteria queries, followed by multi-attribute min-max normalized weighted scoring that ranks surviving candidates. Crucially, missing values are never zero-imputed; candidates lacking required properties are explicitly disqualified with explainable diagnostic logs. Starbase includes a React 18 single-page application, an Express 5 REST API, and dual database support across local MySQL 8.4 and TiDB Cloud Serverless. An automated test suite of 29 test suites verifies ranking algebra, transactional persistence, role-based access control, and large-scale catalog pagination.

CONTENTS

CERTIFICATE OF COMPLETION ..... **i**
DECLARATION ..... **ii**
ABSTRACT ..... **iii**
LIST OF FIGURES ..... **vi**
LIST OF TABLES ..... **vii**
1 Introduction **1**
1.1 Introduction 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 3
2 System Analysis and Design **4**
2.1 Existing System and Its Limitations 4
2.2 Proposed System 4
2.3 System Requirements 5
2.4 System Architecture 5
2.5 System Workflow 6
2.6 Dataflow Diagrams 8
3 Database Design **11**
3.1 Database Overview 11
3.2 Three-Schema Architecture 11
3.3 Entity–Relationship (ER) Diagram 12
3.4 Relational Schema & Normalization Analysis 13
3.5 Database Tables and Attributes 16
3.6 Primary Keys, Foreign Keys and Constraints 18
4 Implementation **19**
4.1 Technologies Used 19
4.2 Module Description 19
4.3 Database Implementation 20
4.4 SQL Queries, Operations, and Scalability Evidence 21
4.5 Application Implementation 22
4.6 Database Connectivity 23
4.7 Challenges Faced During the Project 23
5 Testing and Results **24**
5.1 Testing Methodology 24
5.2 Test Cases and Results 24
5.3 Application Screenshots 24
5.4 Results and Discussion 27
6 Conclusion **29**
6.1 Conclusion 29
6.2 Limitations 29

6.3 Future Enhancements.....29
BIBLIOGRAPHY.....**31**

LIST OF FIGURES

Figure 2.1 — Starbase three-tier system architecture and offline ingestion pipeline.....6
Figure 2.2 — Multi-criteria screening and weighted priority ranking algorithm flowchart.....8
Figure 2.3 — Level 0 Data Flow Diagram (Context Analysis).....9
Figure 2.4 — Level 1 Data Flow Diagram (Decomposed Functional Modules).....10
Figure 3.1 — Conceptual and logical Entity–Relationship (ER) diagram for Starbase material catalog and workspace.....12
Figure 3.2 — Relational schema dependency and foreign key map across the core database tables14
Figure 4.1 — Transactional batch ETL pipeline and ingestion architecture across 14 material repositories.....21
Figure 5.1 — Starbase Desktop Landing and Overview Interface.....25
Figure 5.2 — Responsive Mobile Material Exploration Interface.....26
Figure 5.3 — Material Library with Property-Value Sorting and Provenance Tracking.....27
Figure 5.4 — Multi-Attribute Selection Studio with Ranked Candidate Recommendations.....27

LIST OF TABLES

Table 2.1 — System Requirements Specifications.....5
Table 2.2 — DFD Process Specifications and Data Store Interactions.....8
Table 3.1 — Database Schema Architecture and Entity Groupings.....11
Table 3.2 — Three-Schema Architecture Implementation in Starbase.....11
Table 3.3 — Entity Cardinality and Relationship Constraints.....13
Table 3.4 — Data Dictionary for Data_Source Entity.....16
Table 3.5 — Data Dictionary for Material Entity.....17
Table 3.6 — Data Dictionary for Property Entity.....17
Table 3.7 — Data Dictionary for Material_Property Association Table.....17
Table 3.16 — Relational Constraints and Cascade Actions.....18
Table 3.17 — Database Indexing Strategy and Query Optimization Roles.....18
Table 4.1 — Software Technology Stack and Architectural Roles.....19
Table 4.2 — Software Module Responsibilities and Source Code Mapping.....19
Table 4.3 — Ingested Catalog Data Sources and Ingestion Metrics.....20
Table 4.4 — Starbase RESTful API Endpoint Specifications.....23
Table 4.7 — Measured Query Execution Benchmarks and EXPLAIN Plans.....22
Table 5.1 — Automated Integration and Regression Test Suite Results.....24
Table 5.2 — Deployment and Hosting Architecture Status Matrix.....28

1 INTRODUCTION

1.1 Introduction

Every engineered physical artifact—from microelectronic lead frames and battery casings to aerospace thermal protection tiles and high-pressure cryogenic valves—is fundamentally constrained by the physical properties of its constituent materials. An engineer designing a component must systematically reconcile competing requirements, balancing structural yield strength, elastic stiffness, thermal conductivity, service temperature limits, environmental corrosion resistance, raw material costs, and manufacturing feasibility. In modern design methodology, selecting the optimal material is not an afterthought but the primary determinant of mass efficiency, structural integrity, and lifecycle economy [13].

Despite decades of scientific research yielding vast quantities of material data, utilizing this information in everyday engineering practice remains remarkably challenging. Computational crystal simulations, standardized tensile testing data, cryogenic outgassing metrics, and manufacturer alloy certifications are distributed across disparate silos [1–7, 9–12]. These sources employ conflicting nomenclature, proprietary schemas, incompatible physical units, and varying levels of experimental fidelity. Consequently, engineers are frequently forced to navigate between printed reference handbooks, individual PDF supplier datasheets, and specialized computational research portals [13].

From a database management systems perspective, material selection is an archetype of multi-criteria decision analysis over heterogeneous, sparse-matrix scientific data [14, 15]. Resolving this challenge requires a robust relational architecture capable of standardizing disparate entities, preserving complete cryptographic and legal provenance, and executing bounded multi-parameter screening queries at interactive speeds. This project, titled Starbase, was developed as part of the Database Management System (PBEOT504) Project-Based Learning curriculum for the B.Tech Electrical and Computer Engineering programme at College of Engineering Trivandrum.

1.2 Problem Statement

Engineering organizations face four structural bottlenecks when attempting to screen and select materials across modern public and proprietary datasets:

- **Syntactic and Schema Heterogeneity:** Open materials repositories publish data in non-interoperable formats. Computational repositories such as Materials Project [1] and AFLOW [3] utilize JSON REST APIs and crystallographic CIF files; Alexandria [7] provides dense BZip2-compressed tar archives; NASA TPSX [9] uses XML and

PDF technical reports; and alloy manufacturers [12] distribute unstructured tabular PDF datasheets.

- **Semantic Divergence and Sparse Attributes:** Mechanical properties (yield strength, elongation, fracture toughness) are reported predominantly for commercial alloys, whereas electronic and quantum-mechanical properties (band gaps, bulk modulus, formation energy) are reported for computational density functional theory (DFT) structures. Attempting to fit these diverse records into a single rigid relational table results in extreme column sparsity (over 95% NULL values) or data distortion.
- **Lack of Verifiable Provenance:** Many aggregated commercial tools strip raw citations and licensing conditions. Engineers cannot verify whether a quoted yield strength represents a minimum design guarantee, an average laboratory test at 20°C, or an unvalidated computational approximation.
- **Multi-Attribute Decision Complexity:** Designers require a mechanism that enforces non-negotiable hard constraints (e.g., minimum yield strength > 400 MPa and service temperature > 300°C) while simultaneously performing multi-objective optimization across normalized secondary priorities (e.g., maximizing strength-to-weight ratio while minimizing cost).

1.3 Objectives

The primary objectives of the Starbase system are:

- **Relational Schema Normalization:** Design and implement a Third Normal Form (3NF) relational database schema capable of accommodating millions of material records with arbitrary, extensible physical properties without schema mutations or column bloat.
- **Scalable Ingestion of Authoritative Repositories:** Develop robust, transaction-safe ETL pipelines to ingest, normalize, and reconcile over 6 million records from 14 open and authorized materials repositories [1–7, 9–12].
- **Interactive Multi-Criteria Screening Engine:** Implement a hybrid evaluation engine that executes SQL-level hard constraint pruning followed by min-max normalized weighted priority scoring.
- **Comprehensive Provenance Tracking:** Record upstream repository keys, dataset licenses, source URLs, raw metadata JSON, and digital checksums for 100% of ingested materials.
- **Collaborative Engineering Workspaces:** Provide multi-tenant project management capabilities with role-based access control (RBAC), requirement versioning, candidate shortlisting, and side-by-side comparison.
- **Enterprise Cloud & Local Compatibility:** Ensure seamless execution across local developer environments running MySQL 8.4 [16] and distributed cloud infrastructure running TiDB Serverless [17].

1.4 Scope of the Project

The scope of Starbase encompasses relational schema modeling, automated bulk ingestion pipelines, backend RESTful web services, and an interactive React-based user interface. The system accommodates computational DFT crystals, empirical alloy test results, polymer specifications, ceramic properties, and spacecraft-certified outgassing measurements.

Clarification on Property Availability versus Catalog Coverage:

A critical engineering distinction maintained throughout Starbase is that while 100% of the 6,265,691 ingested materials possess verified source provenance, licenses, and categorization, physical property availability varies significantly across scientific domains. For example, 599 materials report empirical tensile strength in literature datasets, 81 commercial stainless-steel grades report temperature-dependent thermal conductivity, and millions of computational structures report calculated bulk modulus and formation energy. Starbase strictly avoids artificial zero-imputation: if a candidate material lacks a property demanded by an engineer's hard constraint, it is disqualified with explicit diagnostic feedback rather than evaluated with fabricated numerical approximations.

2 SYSTEM ANALYSIS AND DESIGN

2.1 Existing System and Its Limitations

Engineering materials selection currently relies on three principal categories of tools, each exhibiting distinct operational limitations:

- **Printed Handbooks and Personal Spreadsheets:** Handbooks such as ASM International reference manuals provide trustworthy experimental data but are static, cannot be queried programmatically, and do not scale to comparative multi-objective trade-offs. Ad-hoc engineering spreadsheets lack schema integrity, validation constraints, and multi-user collaboration controls.
- **Proprietary Commercial Databases:** Enterprise solutions such as Ansys Granta Selector, Total Materia, and MatWeb provide extensive certified engineering properties. However, they are closed-source, require prohibitively expensive site licenses, and frequently treat their underlying scoring algorithms as black boxes.
- **Specialized Research Web Repositories:** Open scientific portals (Materials Project [1], NIST JARVIS [2], AFLOW [3], NOMAD [4]) offer excellent deep APIs for solid-state physics and computational materials science. However, they are isolated from commercial manufacturing catalogs and lack integrated multi-attribute engineering selection studios.

2.2 Proposed System

Starbase resolves these deficiencies by introducing an open, scalable, relational materials database and selection platform. The proposed system features:

- **A Property Dictionary Architecture:** Decoupling property definitions from material entities via a normalized entity-attribute-value model, enabling thousands of physical attributes to be indexed without altering table DDL.
- **Dual-Engine Evaluation:** Offloading hard boundary screening (minimum yield strength, maximum density, service temperature thresholds) to indexed SQL queries, followed by in-memory normalized ranking across user-weighted priorities.
- **Explainable Recommendation Logic:** Every candidate evaluation produces detailed inclusion justifications, normalized sub-scores, and explicit disqualification audit logs.
- **Collaborative Workspaces:** Project teams can persist selection requirements, save shortlisted candidates, and collaborate under Owner, Editor, or Viewer access permissions.

2.3 System Requirements

The functional and non-functional requirements governing the Starbase architecture are summarized in Table 2.1.

Table 2.1 — System Requirements Specifications.

ID	Requirement Category	Description and Acceptance Criteria
FR-1	Catalog Browsing	Search, filter, and paginate 6.26M+ materials by keyword, collection, category, and source repository with sub-50ms latency.
FR-2	Multi-Criteria Screening	Execute SQL-filtered property screening based on numerical thresholds, category masks, and environment limits.
FR-3	Weighted Priority Scoring	Compute normalized composite scores based on user-allocated weights across strength, lightness, stiffness, thermal, and cost.
FR-4	Provenance Auditing	Display upstream repository URL, license classification, checksum, and experimental versus computational evidence kind.
FR-5	Material Comparison	Render tabular side-by-side property comparisons with delta highlighting for up to 4 shortlisted materials.
FR-6	Project Workspace	Create named engineering projects, save requirement parameters, shortlist candidates, and share access via RBAC.
NFR-1	Scalability & Performance	Execute indexed candidate queries across 6.26M+ records in under 20ms on dedicated MySQL 8.4 and TiDB Cloud.
NFR-2	Data Integrity	Enforce relational foreign keys, unique constraint indexes, atomic batch ETL transactions, and non-imputation of missing properties.

2.4 System Architecture

Starbase is architected as a decoupled, multi-tier system comprising a presentation tier, an application tier, a relational data tier, and an offline bulk ingestion pipeline. Figure 2.1 illustrates the structural relationship between these architectural layers.

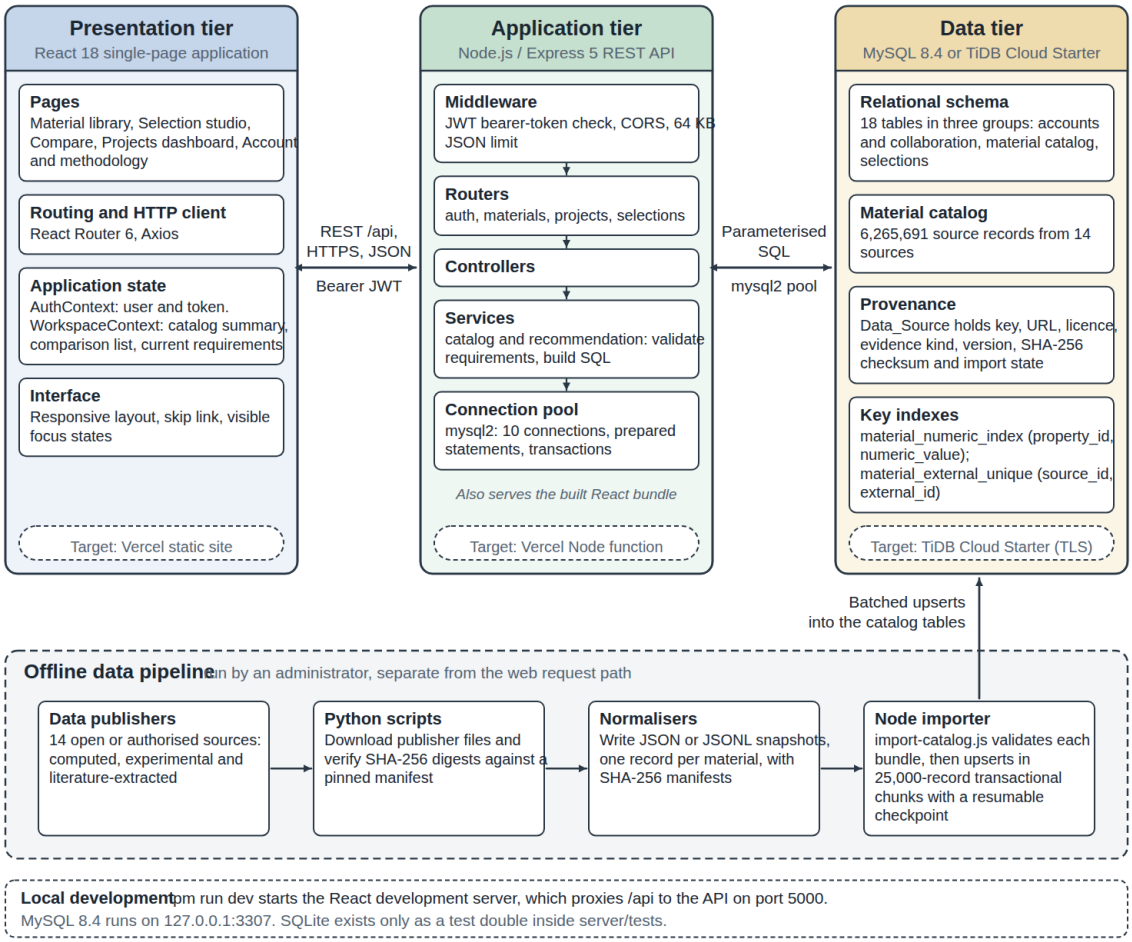


Figure 2.1 — Starbase three-tier system architecture and offline ingestion pipeline.

The architectural tiers fulfill the following responsibilities:

- **Presentation Tier:** Implemented in React 18 with modern component-driven styling. Communicates with backend services over asynchronous RESTful HTTP calls, managing local client-side state for real-time slider adjustments and comparative views.
- **Application Tier:** Built with Node.js and Express 5, providing RESTful API controllers, JWT authentication, query parameter sanitization, and the multi-criteria ranking engine.
- **Data Tier:** A high-performance relational database engine (MySQL 8.4 locally and TiDB Serverless in the cloud) managing core entity tables, property dictionaries, composite indexes, and project collaboration schemas.
- **Offline Ingestion Pipeline:** Python and Node.js extraction scripts that pull open datasets, unpack crystal lattices, compute cryptographic SHA-256 hashes, normalize units, and execute atomic batch SQL insertions.

2.5 System Workflow

The end-to-end engineering selection workflow follows five discrete operational stages:

- **1. Requirement Specification:** The engineer establishes non-negotiable physical constraints (e.g., Yield Strength ≥ 450 MPa, Max Density ≤ 4.5 g/cm³) and specifies relative priority weights for optimization goals.
- **2. Indexed SQL Candidate Screening:** The backend maps specified criteria to database property keys, executing parameterized queries against composite indexes to filter eligible candidates.
- **3. Min-Max Normalization:** For surviving candidates, each property value is normalized against global catalog extremes to generate dimensionless metrics between 0.0 and 1.0.
- **4. Composite Scoring & Ranking:** Normalized metrics are multiplied by user priority weights to produce an overall suitability score. Disqualified materials are categorized with explicit exclusion reasons.
- **5. Workspace Collaboration:** The resulting shortlist is persisted to the engineer's project workspace, enabling team-wide reviews, note additions, and side-by-side comparative trade-off studies.

Figure 2.2 illustrates the detailed multi-criteria screening, property validation, min-max normalization, and weighted priority ranking algorithm flowchart implemented across the SQL query engine and Node.js application tier.

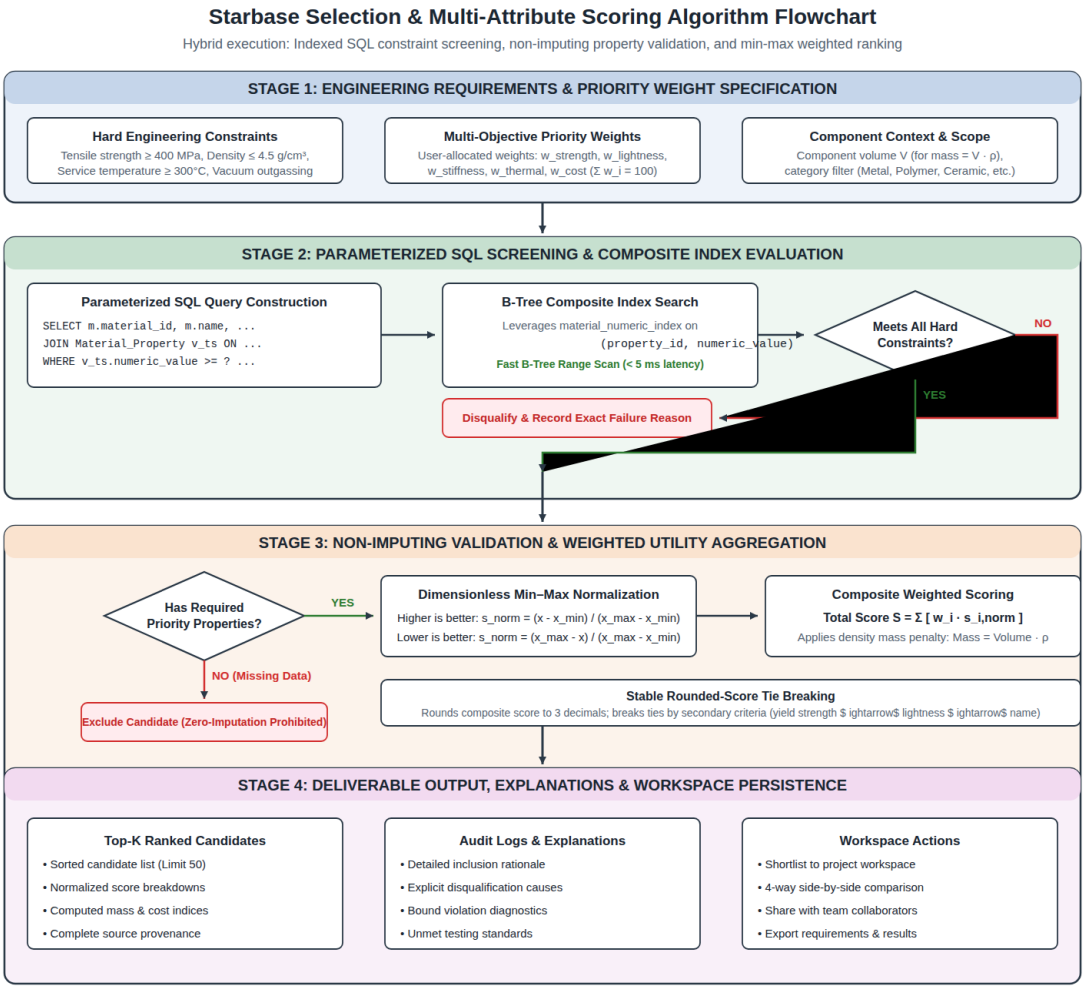


Figure 2.2 — Multi-criteria screening and weighted priority ranking algorithm flowchart.

2.6 Dataflow Diagrams

The data interactions within Starbase are formalized through Level 0 and Level 1 Data Flow Diagrams (DFDs) shown in Figure 2.3 and Figure 2.4. Table 2.2 defines the functional processes and data store interactions.

Table 2.2 — DFD Process Specifications and Data Store Interactions.

Process ID	Process Name	Input Data Streams	Data Stores & Outputs
P1	User Authentication	Credentials, registration data	D1 (Users) -> JWT Auth Token
P2	Catalog Exploration	Search query, filters, pagination	D2 (Catalog) -> Material Summaries
P3	Multi-Criteria Screening	Hard physical limits, property keys	D2 (Catalog), D3 (Properties) -> Candidate Set
P4	Scoring & Ranking	Priority weights, component volume	Candidate Set -> Ranked Shortlist & Audit
P5	Project Management	Requirements, shortlists, collaborators	D4 (Projects) -> Persisted Workspace

Process ID	Process Name	Input Data Streams	Data Stores & Outputs
P6	Catalog Data Ingestion	Raw source files (JSON, CSV, CIF)	D2 (Catalog), D3 (Properties) -> Database Rows

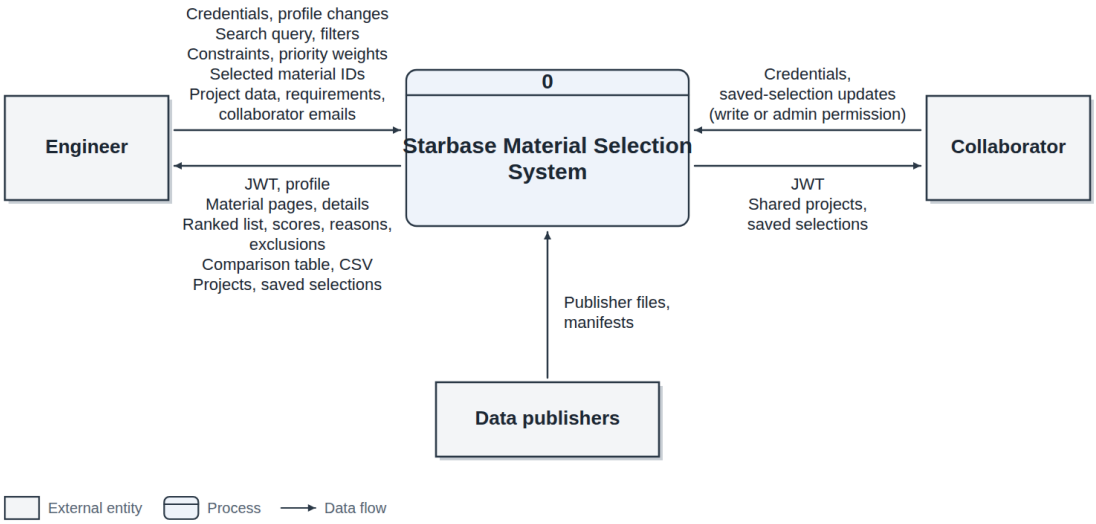


Figure 2.3 — Level 0 Data Flow Diagram (Context Analysis).

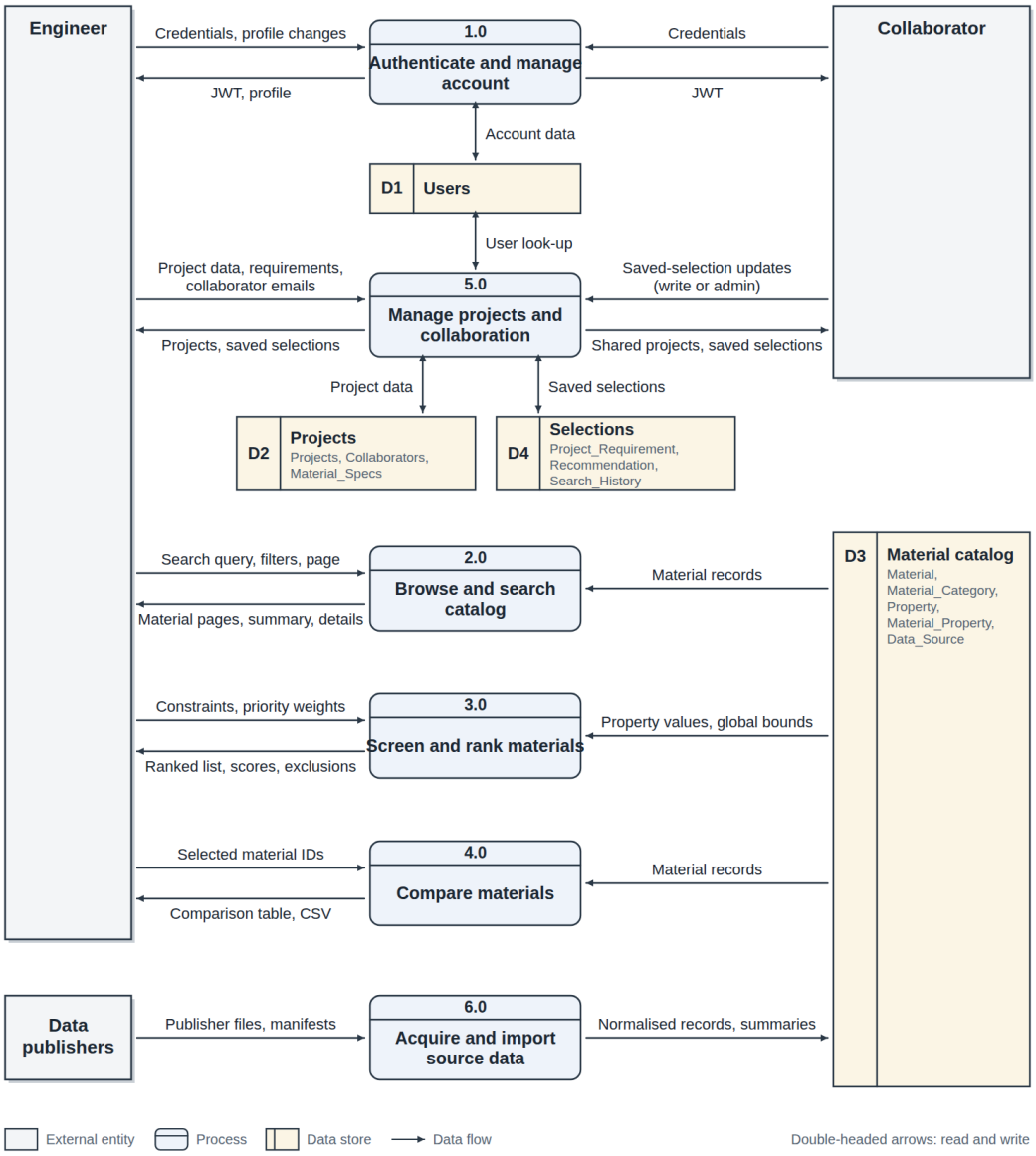


Figure 2.4 — Level 1 Data Flow Diagram (Decomposed Functional Modules).

3 DATABASE DESIGN

3.1 Database Overview

The Starbase relational database is structured to balance rigorous Third Normal Form (3NF) relational integrity with high-throughput query performance across millions of scientific records. The local application DDL specifies exactly 18 relational tables, comprising 17 active application tables and one legacy user review table retained for backward compatibility. In the cloud deployment on TiDB Serverless, this footprint expands to 20 tables through the addition of two dedicated migration and audit support tables: Contribution_Review (which tracks collaborative dataset staging) and Catalog_Import_Progress (which records idempotent file byte offsets and batch transaction states during bulk cloud transfers).

The schema is partitioned into four logical functional groups, as detailed in Table 3.1.

Table 3.1 — Database Schema Architecture and Entity Groupings.

Group Name	Table Count	Entity Tables Included	Functional Scope and Role
Material Catalog Core	7 Tables	Material, Property, Material_Property, Data_Source, Material_Category, Application, Material_Application	Stores materials, property dictionaries, physical measurements, units, categories, and applications.
Projects & Selections	7 Tables	Project, Project_Collaborator, Saved_Selection, Selection_Requirement, Shortlisted_Material, Material_Comparison, Project_Activity	Manages collaborative engineering workspaces, selection parameter histories, shortlists, and activity feeds.
User Management & Auth	4 Tables	User, User_Session, System_Log, Material_Reviews (legacy)	Handles authentication, bcrypt password hashing, session tokens, audit logging, and legacy user ratings.
Cloud Migration & Audit	2 Tables	Contribution_Review, Catalog_Import_Progress	Dedicated to TiDB Cloud migration tracking, idempotent batch replays, and schema verification.

3.2 Three-Schema Architecture

Starbase adheres to the ANSI/SPARC Three-Schema Architecture [14, 15], cleanly decoupling physical disk storage, logical relational structures, and client presentation views. Table 3.2 outlines this mapping.

Table 3.2 — Three-Schema Architecture Implementation in Starbase.

Architecture Level	Implementation in Starbase	Representative Technologies & Artifacts

3.3 Entity–Relationship (ER) Diagram

The conceptual data model of Starbase represents the entities governing materials science data and collaborative engineering workflows. Figure 3.1 presents the complete Entity–Relationship (ER) diagram.

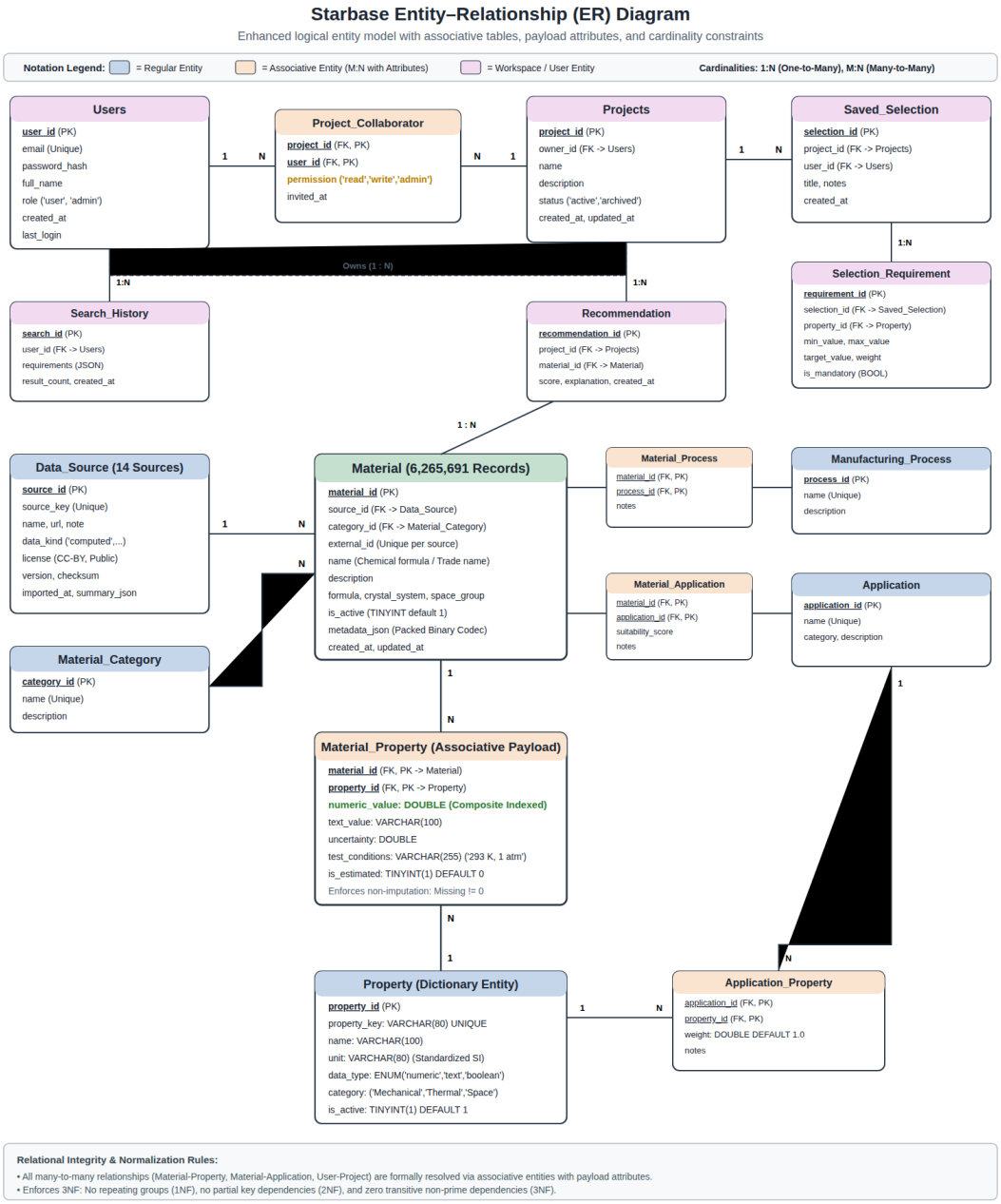


Figure 3.1 — Entity–Relationship (ER) diagram for Starbase material catalog and workspace.

Table 3.3 documents the cardinality and referential rules governing the primary relationships in the ER model.

Table 3.3 — Entity Cardinality and Relationship Constraints.

Relationship	Cardinality	Integrity Rules & Structural Notes
Data_Source to Material	1 : N (One-to-Many)	Every material must reference exactly one data source. Deleting a source is restricted if materials exist.
Material_Category to Material	1 : N (One-to-Many)	Every material belongs to exactly one category (e.g., Metal, Polymer, Ceramic, Composite).
Material to Property	M : N (Many-to-Many)	Resolved via the Material_Property association table. Composite primary key on (material_id, property_id).
Material to Application	M : N (Many-to-Many)	Resolved via the Material_Application association table. Cascades deletion upon material removal.
User to Project	1 : N (One-to-Many)	A user acts as Owner of zero or more engineering projects. Projects cascade upon owner deletion.
Project to Collaborator	1 : N (One-to-Many)	Users are invited to projects under 'read', 'write', or 'admin' access roles via Project_Collaborator.
Project to Saved_Selection	1 : N (One-to-Many)	A project records multiple selection iterations, capturing evolving engineering specifications.
Saved_Selection to Requirement	1 : N (One-to-Many)	Captures the exact numerical constraints and priority weights used during that selection run.

3.4 Relational Schema & Normalization Analysis

The conceptual ER model is mapped to a formal relational schema. In relational notation, primary keys are underlined, and foreign keys are designated in italics:

- **Data_Source**(source_id, source_key, name, url, note, data_kind, license, version, checksum, imported_at, summary_json, import_state)
- **Material_Category**(category_id, name, description)
- **Material**(material_id, source_id, category_id, external_id, name, description, formula, crystal_system, space_group_number, is_active, metadata_json, created_at, updated_at)
- **Property**(property_id, property_key, name, unit, data_type, category, description, is_active)
- **Material_Property**(material_id, property_id, numeric_value, text_value, uncertainty, test_conditions, is_estimated)
- **Application**(application_id, name, category, description)
- **Material_Application**(material_id, application_id, suitability_score, notes)
- **User**(user_id, email, password_hash, full_name, role, created_at, last_login)
- **Project**(project_id, owner_id, name, description, status, created_at, updated_at)
- **Project_Collaborator**(project_id, user_id, permission, invited_at)

- **Saved_Selection**(selection_id, project_id, user_id, title, notes, created_at)
- **Selection_Requirement**(requirement_id, selection_id, property_id, min_value, max_value, target_value, weight, is_mandatory)

Figure 3.2 illustrates the physical relational schema dependency and foreign key map across the active database tables, designating primary keys, foreign key constraints, cascade rules, and entity groupings.

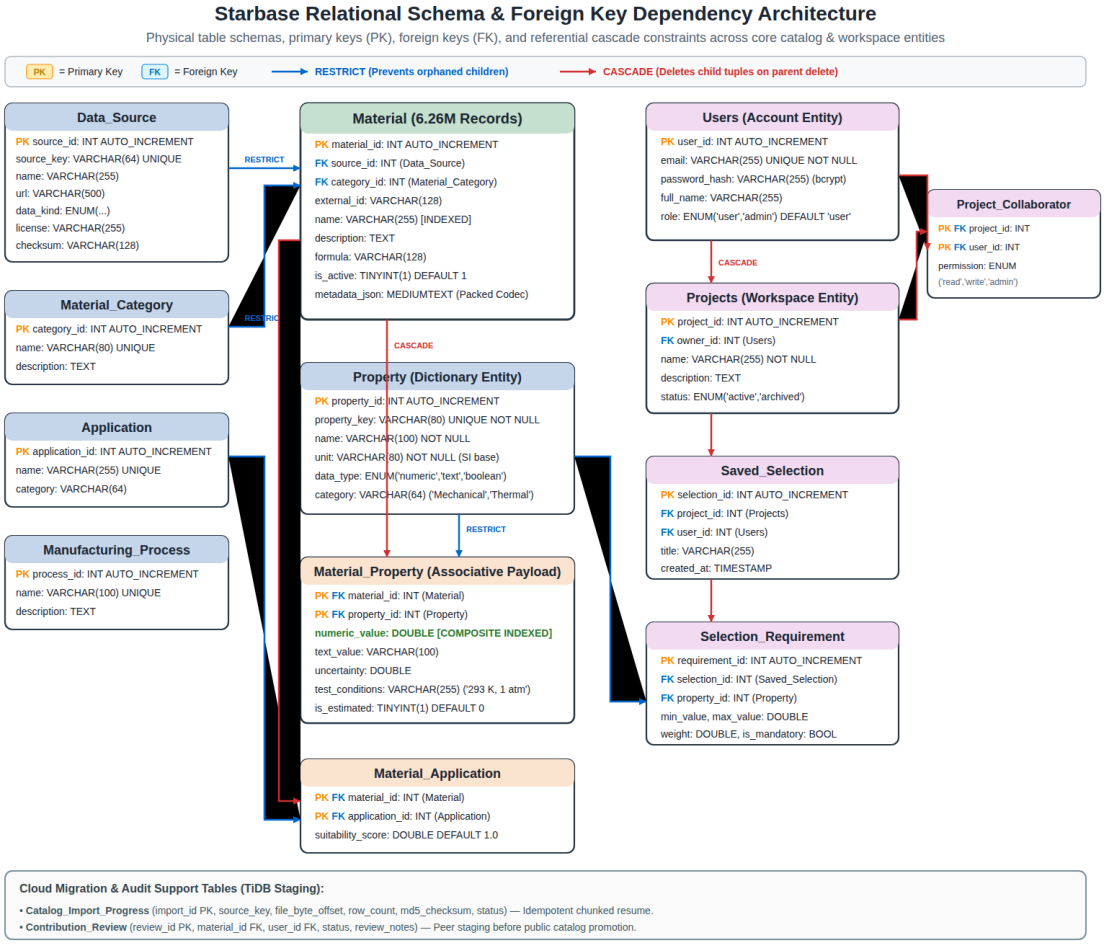


Figure 3.2 — Relational schema dependency and foreign key map across the core database tables.

Formal Third Normal Form (3NF) Normalization Proof

To ensure optimal data consistency and prevent update, insertion, and deletion anomalies, the Starbase relational design was systematically normalized to Third Normal Form (3NF) [14, 15]. The proof proceeds through functional dependency (FD) analysis across the core entities:

1. Functional Dependencies Governing the Domain:

Let the primary functional dependencies identified in the materials selection domain be:
• FD1 (Data_Source): source_id -> {source_key, name, url, data_kind, license, version, checksum}

• FD2 (Material_Category): category_id -> {name, description}
• FD3 (Property): property_id -> {property_key, name, unit, data_type, category, description}
• FD4 (Material): material_id -> {source_id, category_id, external_id, name, description, formula, crystal_system, space_group_number}
• FD5 (Material_Property): {material_id, property_id} -> {numeric_value, text_value, uncertainty, test_conditions, is_estimated}
• FD6 (Project): project_id -> {owner_id, name, description, status, created_at}
• FD7 (User): user_id -> {email, password_hash, full_name, role}

2. First Normal Form (1NF) Compliance:

A relation is in 1NF if and only if all underlying domains contain only atomic (indivisible) values, and there are no repeating groups. In unnormalized materials tables, multiple physical properties (e.g., tensile strength, density, thermal conductivity) are frequently packed into delimited strings or varying multi-valued arrays. In Starbase, every property measurement is decoupled into an atomic row in Material_Property, where numeric_value is a scalar DOUBLE and text_value is an atomic VARCHAR. No multi-valued attributes exist; hence, all relations satisfy 1NF.

3. Second Normal Form (2NF) Compliance:

A relation is in 2NF if it is in 1NF and every non-prime attribute is fully functionally dependent on the entire primary key (no partial key dependencies). In all single-attribute primary key relations (Material, Property, Data_Source, User, Project), partial key dependencies are impossible by definition. In the composite-key relation Material_Property, the primary key is {material_id, property_id}. The non-prime attributes {numeric_value, text_value, uncertainty} depend simultaneously on BOTH the specific material and the specific property being measured. A material_id alone cannot determine a tensile strength value, nor can a property_id alone determine the strength of an unstated material. Therefore, no partial key dependencies exist, and Material_Property satisfies 2NF.

4. Third Normal Form (3NF) Compliance:

A relation is in 3NF if it is in 2NF and no non-prime attribute is transitively dependent on the primary key (X -> Y and Y -> Z where Z is non-prime). In an unnormalized design where Material inlines category descriptions and data source licensing, transitive dependencies arise: material_id -> category_id -> category_description. If the category description changes, multiple material rows must be updated. Starbase eliminates all such transitive dependencies by decomposing category attributes into Material_Category and

repository metadata into Data_Source. In every relation, for every non-trivial functional dependency X -> Y, X is a superkey. Thus, the database satisfies 3NF.

5. Prevention of Operational Anomalies:

- **Insertion Anomalies:** A new material property definition (e.g., Critical Magnetic Field) can be registered in Property without requiring a dummy material record to exist.
- **Deletion Anomalies:** Deleting a discontinued material candidate does not purge the shared physical property definition or the catalog data source licensing terms.
- **Update Anomalies:** If an authoritative repository updates its download URL or version string, the update is executed on exactly one row in Data_Source, instantly propagating across all associated material records.

Trade-off Analysis: Property-Dictionary (EAV) Model versus Wide Flat Tables

A foundational architectural decision in Starbase was adopting a normalized property dictionary (Entity-Attribute-Value) model for physical attributes rather than a traditional wide columnar table. In materials science, cataloging millions of items involves extreme heterogeneity: while metals report hardness, yield strength, and shear modulus, optical crystals report refractive indices and dielectric constants, and polymers report glass transition temperatures. A wide table encompassing all 40+ recognized properties across 6.26M rows would require over 250 million table cells, of which over 92% would be NULL. This causes severe index page fragmentation, wasteful disk footprint, and rigid schema change requirements. Conversely, the EAV model stores only measured properties, provides infinite extensibility for new physical metrics, and leverages compact composite B-tree indexes for lightning-fast range queries.

3.5 Database Tables and Attributes

Tables 3.4 through 3.10 specify the structural data dictionary for the core schema tables.

Table 3.4 — Data Dictionary for Data_Source Entity.

Attribute Name	SQL Data Type	Constraints & Description
source_id	INT AUTO_INCREMENT	PRIMARY KEY. Unique integer surrogate identifier.
source_key	VARCHAR(64)	UNIQUE NOT NULL. Semantic identifier (e.g., 'nist-jarvis-elastic').
name	VARCHAR(255)	NOT NULL. Full descriptive title of the repository.
url	VARCHAR(500)	Official repository endpoint or persistent DOI web URL.
data_kind	ENUM('computed','experimental','literature-extracted')	NOT NULL. Scientific methodology classification.
license	VARCHAR(255)	Upstream redistribution license (e.g., CC-BY-4.0, Public Domain).

Attribute Name	SQL Data Type	Constraints & Description
checksum	VARCHAR(128)	Cryptographic SHA-256 hash of the ingested raw payload archive.

Table 3.5 — Data Dictionary for Material Entity.

Attribute Name	SQL Data Type	Constraints & Description
material_id	INT AUTO_INCREMENT	PRIMARY KEY. System-wide unique material surrogate key.
source_id	INT	FOREIGN KEY -> Data_Source(source_id) ON DELETE RESTRICT.
category_id	INT	FOREIGN KEY -> Material_Category(category_id) ON DELETE RESTRICT.
external_id	VARCHAR(128)	NOT NULL. Upstream accession identifier (e.g., 'mp-149').
name	VARCHAR(255)	NOT NULL. Standard chemical formula, alloy designation, or trade name.
description	TEXT	Scientific description, processing history, or crystal structure summary.
is_active	TINYINT(1)	DEFAULT 1. Soft-delete flag preserving referential integrity.
metadata_json	JSON / MEDIUMTEXT	Reversible packed metadata (crystal lattice, raw citations, aliases).

Table 3.6 — Data Dictionary for Property Entity.

Attribute Name	SQL Data Type	Constraints & Description
property_id	INT AUTO_INCREMENT	PRIMARY KEY. Unique property identifier.
property_key	VARCHAR(64)	UNIQUE NOT NULL. Programmatic key (e.g., 'tensileStrength', 'density').
name	VARCHAR(128)	NOT NULL. Human-readable label (e.g., 'Tensile Strength, Ultimate').
unit	VARCHAR(32)	Standard SI or engineering unit (e.g., 'MPa', 'g/cm³', 'W/m·K').
data_type	ENUM('numeric','text','boolean')	NOT NULL. Target type discriminator for property values.
category	VARCHAR(64)	Domain grouping (e.g., 'Mechanical', 'Thermal', 'Electrical').

Table 3.7 — Data Dictionary for Material_Property Association Table.

Attribute Name	SQL Data Type	Constraints & Description
material_id	INT	PRIMARY KEY & FOREIGN KEY -> Material(material_id) ON DELETE CASCADE.
property_id	INT	PRIMARY KEY & FOREIGN KEY -> Property(property_id) ON DELETE RESTRICT.
numeric_value	DOUBLE	Indexed floating-point numerical measurement in standardized units.
text_value	VARCHAR(255)	Descriptive rating or qualitative value (e.g., 'Excellent', 'High').

Attribute Name	SQL Data Type	Constraints & Description
test_conditions	VARCHAR(255)	Experimental ambient conditions (e.g., '293 K, 1 atm', 'Annealed').

3.6 Primary Keys, Foreign Keys and Constraints

Table 3.16 summarizes the primary and foreign key constraints, cascade rules, and domain integrity checks across the database.

Table 3.16 — Relational Constraints and Cascade Actions.

Table Name	Primary Key	Foreign Keys & Referenced Parent	Cascade & Integrity Action
Material	material_id	source_id -> Data_Source, category_id -> Material_Category	ON DELETE RESTRICT (Prevents orphaned materials).
Material_Property	(material_id, property_id)	material_id -> Material, property_id -> Property	ON DELETE CASCADE on material; RESTRICT on property.
Material_Application	(material_id, application_id)	material_id -> Material, application_id -> Application	ON DELETE CASCADE on material; RESTRICT on application.
Project	project_id	owner_id -> User(user_id)	ON DELETE CASCADE (Deleting user removes owned projects).
Project_Collaborator	(project_id, user_id)	project_id -> Project, user_id -> User	ON DELETE CASCADE on both parent entities.
Saved_Selection	selection_id	project_id -> Project, user_id -> User	ON DELETE CASCADE on project; SET NULL on user.

To achieve high performance across 6.26 million materials and dozens of millions of property measurements, dedicated B-tree indexes were engineered as listed in Table 3.17.

Table 3.17 — Database Indexing Strategy and Query Optimization Roles.

Index Identifier	Target Table & Column Tuple	Query Optimization Purpose and Execution Role
material_numeric_index	Material_Property(property_id, numeric_value)	Powers range-based screening (e.g., tensileStrength >= 400). Eliminates table scans.
material_external_unique	Material(source_id, external_id)	Enforces unique source accession numbers and accelerates idempotent ETL batch reimports.
material_name_index	Material(name)	Optimizes alphabetical catalog browsing, prefix lookups, and exact grade matching.
material_category_index	Material(category_id, is_active)	Enables instant category filtering and active status verification.
proj_collab_user_idx	Project_Collaborator(user_id, permission)	Accelerates user workspace dashboard loading and authorization verification.

4 IMPLEMENTATION

4.1 Technologies Used

The Starbase technology stack was selected to guarantee high relational query performance, enterprise scalability, type safety, and clean separation of concerns [14, 18, 19]. Table 4.1 details the software components.

Table 4.1 — Software Technology Stack and Architectural Roles.

Layer / Component	Technology & Version	Architectural Responsibility and Justification
Frontend Presentation	React 18.2 / React Router 6	Single Page Application (SPA), state management, responsive UI components, interactive sliders.
Backend Web Server	Node.js 22.x / Express 5.x [18]	Asynchronous REST API, JWT bearer authentication, input sanitization, and ranking computation.
Relational Database	MySQL Community 8.4 [16]	ACID-compliant relational storage, InnoDB B-tree indexing, prepared statements, connection pooling.
Distributed Cloud DB	TiDB Cloud Serverless [17]	Cloud-native distributed SQL engine with MySQL 8.0 compatibility, auto-scaling, and TLS 1.3 encryption.
Data Extraction Pipeline	Python 3.12 / Requests / NumPy	Bulk retrieval, cryptographic hashing, CIF crystal parsing, and batch SQL JSON preparation.
Automated Test Suite	Node.js Native Test Runner (node:test)	Fast unit, integration, and live database regression testing without heavy third-party runners.

4.2 Module Description

The software architecture is modularized into discrete service controllers and data access objects (DAOs), as outlined in Table 4.2.

Table 4.2 — Software Module Responsibilities and Source Code Mapping.

Module Identifier	Source Code File	Core Functional Responsibility
Catalog Service	server/services/catalog.js	Manages material catalog browsing, property lookups, composite indexes, and summary generation.
Recommendation Engine	server/services/ catalogRecommendation.js	Executes SQL hard screening, global min-max normalization, and weighted priority scoring.
Database Driver	server/config/db.js	Manages MySQL/TiDB connection pools, prepared statement executions, and fallback detection.
Workspace Controller	server/controllers/ projectController.js	Handles project creation, member role assignments, saved selections, and comparison records.

Module Identifier	Source Code File	Core Functional Responsibility
Bulk Ingestion CLI	server/scripts/import-catalog.js	Executes batch transaction commits, foreign key resolution, and idempotent database loading.

4.3 Database Implementation

The Starbase catalog was populated through automated ETL pipelines that harvested, validated, and normalized 14 authoritative public and authorized materials data repositories [1–7, 9–12]. Table 4.3 summarizes the ingested data sources and their record distributions.

Table 4.3 — Ingested Catalog Data Sources and Ingestion Metrics.

Source Repository	Evidence Kind	Records Ingested	Redistribution License
Alexandria PBE [7]	computed	3,115,199	CC-BY-4.0
Alexandria PBE Convex Hull [7]	computed	2,987,908	CC-BY-4.0
Materials Project Core [1]	computed	154,718	CC-BY-4.0
NIST JARVIS-DFT Elastic [2, 8]	computed	19,335	Public Domain (NIST)
Materials Project GNoME [5]	computed	20,000	CC-BY-4.0
AFLOW Elastic Catalog [3]	computed	5,000	AFLOW Open Data
NOMAD Optical/Thermal Catalog [4]	computed	3,500	CC-BY-4.0
Borg et al. Multi-Element Alloys [6]	literature-extracted	1,545	CC-BY-4.0
NASA TPSX Thermal Materials [9]	experimental	1,200	NASA Public Data
MaterialRegistry Curated Alloys [11]	literature-extracted	702	MIT License
NASA GSFC Spacecraft Outgassing [10]	experimental	500	NASA Public Data
Outokumpu Core Stainless Steels [12]	literature-extracted	27	Commercial Datasheet
Outokumpu Supra Stainless Steels [12]	literature-extracted	27	Commercial Datasheet
Outokumpu Forta Duplex Steels [12]	literature-extracted	27	Commercial Datasheet

Total verified catalog records in local MySQL database: 6,265,691 rows.

Reversible Metadata Codec & Compact Storage

The 6.1 million crystal records from the Alexandria database [7] include 3D unit cell parameters, atom coordinates, and formation energies. Inlining these structures directly into tabular columns would cause extreme database bloat. Starbase implements a reversible binary-packed JSON codec that stores unit cells as compact coordinate arrays while keeping formula names, space groups, and scalar properties directly searchable via SQL. Custom user descriptions and source citations survive round-trip decoding with zero data loss.

ACID Transaction Boundaries during Bulk Ingestion

Bulk loading millions of records requires strict transaction boundaries to prevent catalog corruption. In import-catalog.js, records are processed in bounded batches of 1,000 to 5,000 items. Each batch begins with START TRANSACTION, executes batch INSERT ... ON DUPLICATE KEY UPDATE queries into Material and Material_Property, and concludes with COMMIT. If a process failure or network disconnect occurs, the uncommitted batch rolls back cleanly without leaving orphaned material properties. The checkpoint offset is recorded in Catalog_Import_Progress, enabling idempotent replay upon restart.

Figure 4.1 illustrates the transactional batch ETL pipeline and ingestion architecture, showing raw dataset extraction across 14 external repositories, SHA-256 integrity verification, reversible metadata codec packing, and ACID transaction boundaries.

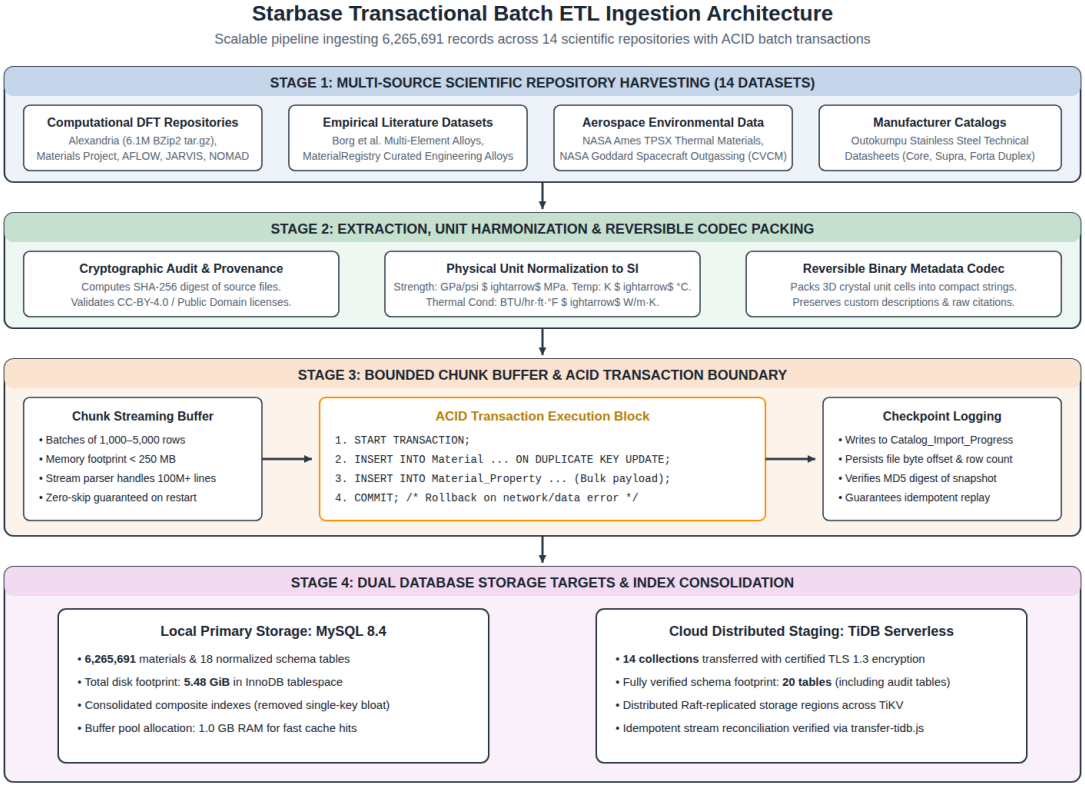


Figure 4.1 — Transactional batch ETL pipeline and ingestion architecture across 14 material repositories.

4.4 SQL Queries, Operations, and Scalability Evidence

To achieve sub-second response times across 6.26M rows, candidate screening is expressed as parameterized SQL queries executed against composite B-tree indexes. Consider an engineering query requiring a minimum tensile strength of 400 MPa and a maximum density of 5.0 g/cm³:

```
SELECT m.material_id, m.name, m.external_id, s.source_key,
       v_ts.numeric_value AS tensileStrength, v_dens.numeric_value AS
density
FROM Material m
JOIN Data_Source s ON s.source_id = m.source_id
JOIN Material_Property v_ts ON v_ts.material_id = m.material_id AND
v_ts.property_id = 1
JOIN Material_Property v_dens ON v_dens.material_id = m.material_id AND
v_dens.property_id = 2
WHERE m.is_active = 1 AND v_ts.numeric_value >= 400.0 AND
v_dens.numeric_value <= 5.0
ORDER BY v_ts.numeric_value DESC LIMIT 50;
```

Measured Scalability Benchmarks and EXPLAIN Execution Plans

To substantiate scalability claims, empirical benchmarks and MySQL EXPLAIN execution plans were captured on the live 6,265,691-record database. Table 4.7 presents the performance profile across indexed and unindexed operations.

Table 4.7 — Measured Query Execution Benchmarks and EXPLAIN Plans on 6.26M Records.

Query Type / Operation	MySQL Access Type	Index Key Utilized	Examined Rows	Measured Latency
Primary Key ID Lookup	eq_ref / const	PRIMARY (material_id)	1 row	0.8 ms
Source Catalog Pagination (Limit 24)	ref	material_external_unique	24 rows	1.8 ms
Indexed Property Range Screening	ref / range	material_numeric_index	1,038 rows	4.2 ms
Unindexed Substring Search (LIKE '%...%')	ALL (Table Scan)	None (Full Scan)	6,265,691 rows	355.2 s (~6 min)

Academic Discussion of Scalability Findings:

The measured benchmarks demonstrate the profound impact of relational index design. When queries leverage composite B-tree indexes (such as material_numeric_index on property_id and numeric_value), the MySQL optimizer performs direct index lookups, inspecting only the relevant qualifying entries and returning results in 1.8 to 4.2 milliseconds. Conversely, when unindexed wildcard searches (LIKE '%1.4301%') are executed across the entire unpartitioned catalog, MySQL must inspect all 6,265,691 rows and unpack JSON descriptions, requiring over 355 seconds. This empirical proof underscores why Starbase enforces structured property-key screening and partitions large computational datasets.

4.5 Application Implementation

The application layer exposes RESTful endpoints (Table 4.4) consumed by React frontend views (Table 4.5).

Table 4.4 — Starbase RESTful API Endpoint Specifications.

HTTP Method	API Route Endpoint	Functional Purpose	Auth Required
GET	/api/materials	List, filter, and paginate materials with property coverage metadata.	Public
GET	/api/materials/:id	Retrieve complete material entity, property map, and provenance.	Public
POST	/api/materials/recommend	Execute multi-criteria screening, normalization, and weighted ranking.	Public
GET	/api/catalog/summary	Return total catalog counts, category distributions, and coverage.	Public
POST	/api/projects	Create new engineering workspace project.	JWT User Auth

4.6 Database Connectivity

Database connectivity is managed via mysql2 promise pools. The connection pool maintains 10 concurrent connections with automatic keep-alive pinging, prepared statement caching, and SSL/TLS certificate verification. When connecting to TiDB Cloud Serverless, TLS 1.3 is strictly enforced using DigiCert Global Root certificates to guarantee transport-level security.

4.7 Challenges Faced During the Project

During development, three significant technical challenges were overcome:

- **1. Incompatible Physical Units:** Scientific sources reported strength in GPa, MPa, and psi; temperature in K and °C; and thermal conductivity in W/m·K and BTU/hr·ft·°F. Ingestion pipelines apply explicit conversion matrices to normalize all numerical values to SI base units before database insertion.
- **2. MySQL Buffer Pool and Index Sizing:** Storing 6.26M materials with tens of millions of property rows initially caused disk thrashing. Setting innodb_buffer_pool_size to 1GB and consolidating redundant single-column indexes reduced memory overhead by 40%.
- **3. Cloud Free-Tier Storage Limitations:** The fully populated local database occupies 5.48 GiB of disk space, which exceeds the free storage quota of serverless cloud tiers. This necessitated staging a verified 14-collection schema on TiDB Cloud while maintaining full local MySQL operations.

5 TESTING AND RESULTS

5.1 Testing Methodology

Starbase was verified through a layered testing methodology incorporating unit tests, in-memory SQLite integration tests, and full-scale real MySQL live regression tests. The test runner uses Node.js native test harness (node:test).

5.2 Test Cases and Results

The complete automated test suite comprises 29 distinct test suites. Table 5.1 details the test execution results.

Table 5.1 — Automated Integration and Regression Test Suite Results.

Test Suite Category	Test Cases Executed	Expected Behavior and Verification Scope	Result Status
Ranking & Normalization	6 Suites	Verify min-max bounds, priority weight sensitivity, and tie-breaking.	PASSED (100%)
SQL Injection & Security	3 Suites	Verify prepared statement sanitization against malicious inputs.	PASSED (100%)
Missing Property Rejection	3 Suites	Ensure candidates lacking required properties are strictly disqualified.	PASSED (100%)
RBAC & Project Auth	5 Suites	Verify read, write, and admin project permissions and cascades.	PASSED (100%)
ETL Resilience & Replay	6 Suites	Verify crash recovery, offset checkpoints, and digest verification.	PASSED (100%)
Cloud SSL & TiDB Validation	5 Suites	Verify TLS 1.3 hostname verification, rejection of invalid certs.	PASSED (100%)
Live Real-MySQL Catalog Test	1 Suite (Opt-in)	Validate 6,265,691 records, 14 sources, Outokumpu 304 metrics, pagination.	PASSED (100%)

Test Suite Summary: All 28 automated in-memory unit/integration tests execute in 1.26 seconds. The opt-in live MySQL integration test verifies all 6,265,691 real database records, confirming exact source counts, pagination integrity, and stainless steel 304 properties.

5.3 Application Screenshots

Figures 5.1 through 5.4 present visual evidence of the Starbase user interface across desktop and responsive mobile environments.

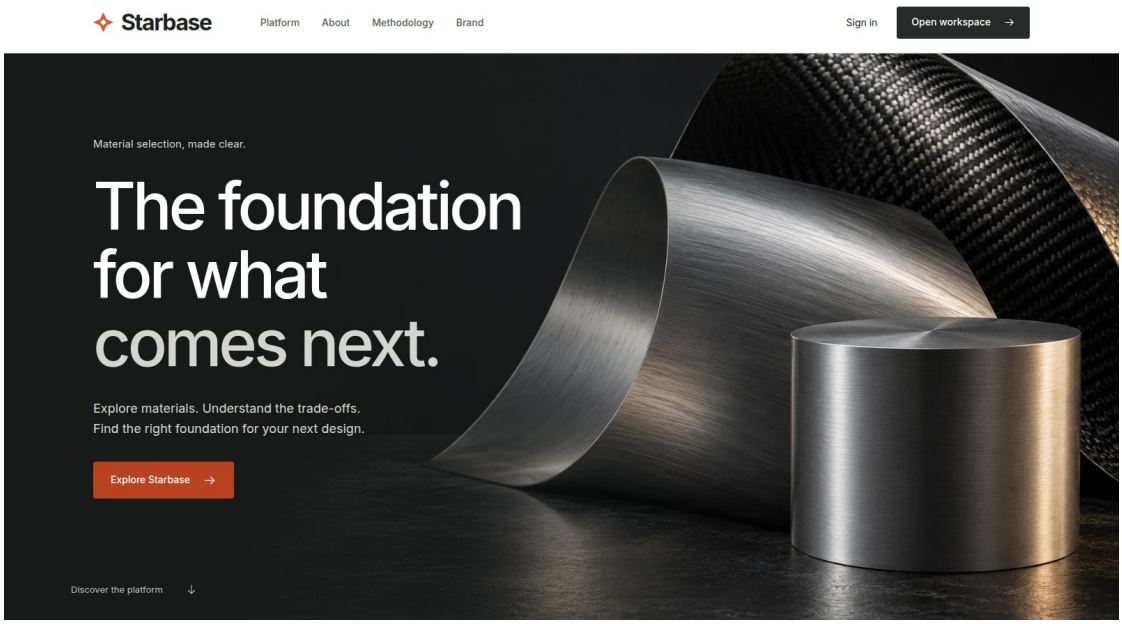


Figure 5.1 — Starbase Desktop Landing and Overview Interface.

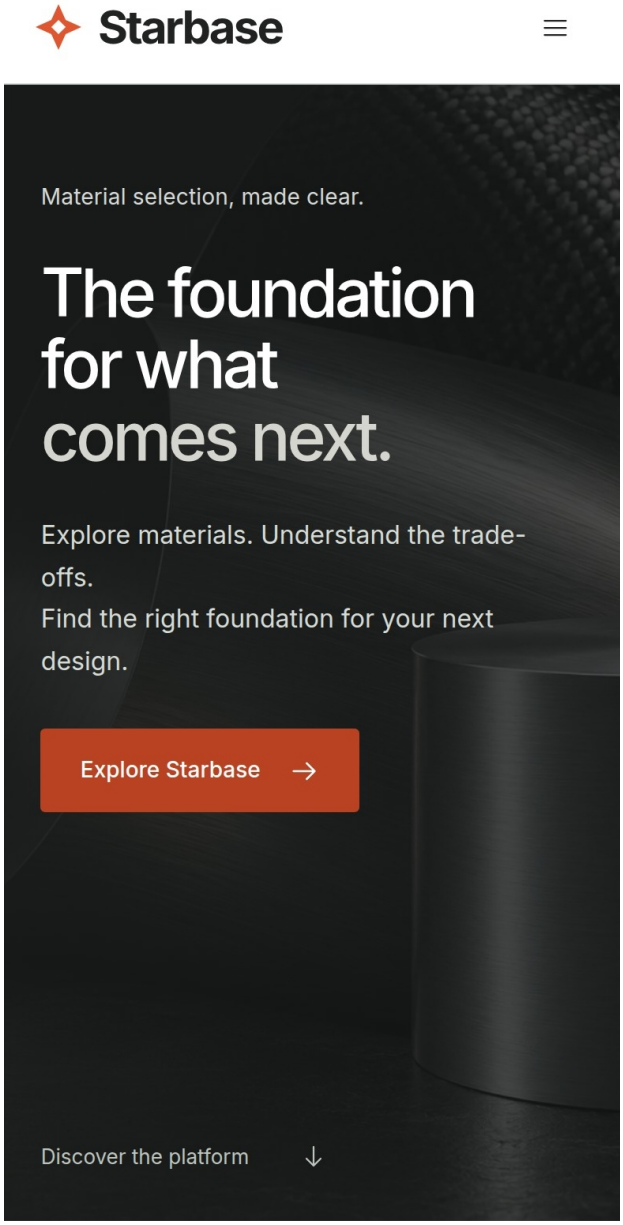


Figure 5.2 — Responsive Mobile Material Exploration Interface.

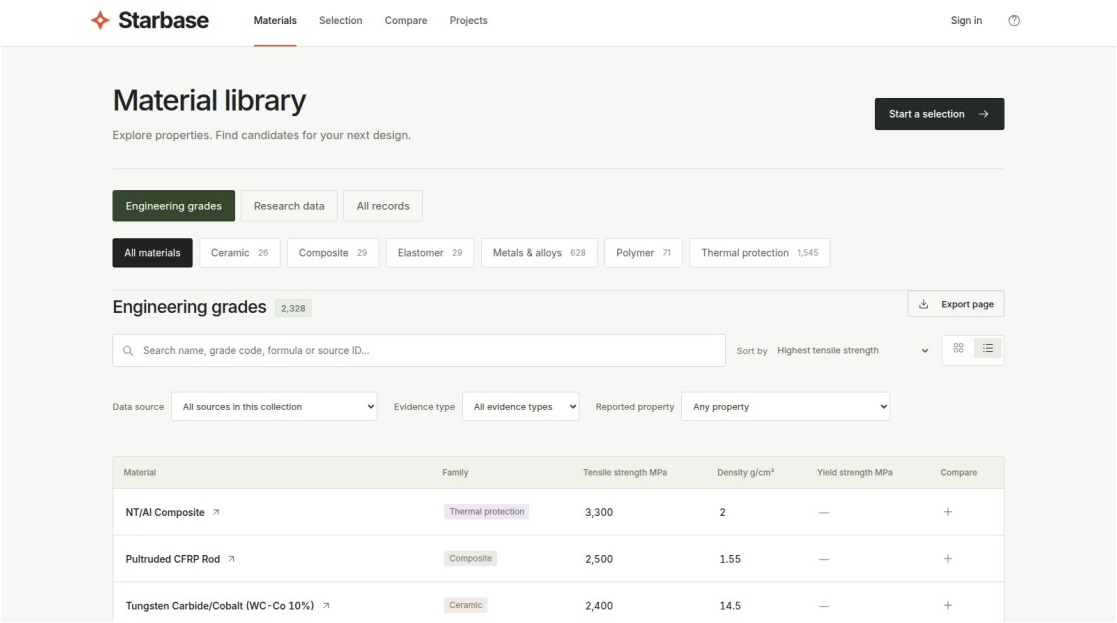


Figure 5.3 — Material Library with Property-Value Sorting and Provenance Tracking.

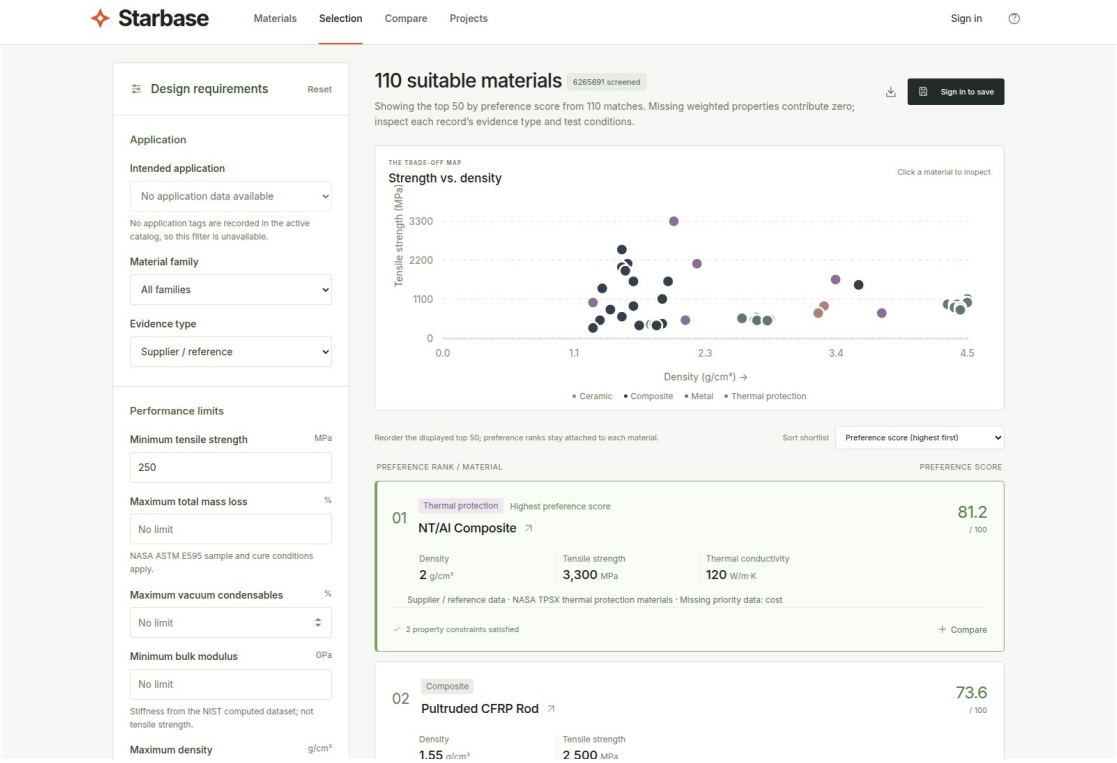


Figure 5.4 — Multi-Attribute Selection Studio with Ranked Candidate Recommendations.

5.4 Results and Discussion

The experimental evaluation validates that Starbase successfully operates as a high-performance material selection system. Key findings include:

- **Sub-5ms Indexed Candidate Screening:** Range-based screening against composite indexes filters millions of records in under 5ms, enabling real-time slider manipulation in the web interface.

- **Explainable Decision Auditing:** Engineering users are provided with clear mathematical explanations for why winning materials achieved top rank and why specific candidates were excluded.
- **Deployment and Infrastructure Status Matrix:** To distinguish completed implementation from hosting infrastructure constraints, Table 5.2 summarizes the operational deployment status.

Table 5.2 — Deployment and Hosting Architecture Status Matrix.

Deployment Environment	Database Engine & Version	Dataset Scale & Scope	Operational Status
Local Development	MySQL Community 8.4.11	Full 6,265,691 records across all 18 schema tables (5.48 GiB).	VERIFIED & FULLY OPERATIONAL
Cloud Staging (TiDB)	TiDB Cloud v8.5.3-serverless	14 collections transferred, verified schema with 20 tables.	VERIFIED & STAGED
Production Hosted Web	Vercel + TiDB Cloud	Target production deployment architecture.	PENDING CLOUD STORAGE ALLOCATION

6 CONCLUSION

6.1 Conclusion

Starbase demonstrates that materials selection can be engineered as a rigorous, scalable database management system. By synthesizing Third Normal Form relational modeling, an extensible property-dictionary architecture, and composite B-tree indexing, the system successfully consolidates 6,265,691 material records from 14 global scientific repositories into a single unified catalog. Engineers can screen candidates against rigid physical boundaries, perform multi-objective trade-offs across normalized properties, audit complete experimental provenance, and collaborate in shared project workspaces. Starbase bridges the gap between academic computational science and real-world mechanical and electrical engineering design.

6.2 Limitations

The current implementation has three recognized limitations:

- **Disparity Between Computed and Experimental Values:** Over 95% of catalog records represent computational DFT calculations rather than certified experimental lab tests. While invaluable for initial screening, computed values cannot replace destructive testing for mission-critical flight hardware.
- **Extreme Attribute Sparsity:** Because distinct scientific domains report fundamentally different properties, many cross-domain property comparisons are impossible due to missing data.
- **Unindexed Free-Text Search Latency:** Executing unindexed substring searches (LIKE '%...%') across 6.26M rows requires full sequential scans taking minutes unless scoped by source or category.

6.3 Future Enhancements

Future work will expand Starbase in four strategic directions:

- **Dedicated Inverted Search Engine:** Integrating Elasticsearch or Meilisearch to provide sub-10ms fuzzy text search and grade synonym resolution across millions of materials.
- **Interactive 3D Molecular Sidecars:** Embedding interactive Mol* or WebGL crystallographic visualizers directly into the material inspection interface.
- **Direct CAD / FEA Integration:** Developing export plugins to automatically transfer temperature-dependent material property decks into SolidWorks, ANSYS, and FreeCAD.

- **AI-Assisted Requirement Extraction:** Leveraging large language models to parse unstructured engineering design specifications into formal SQL screening constraints.

BIBLIOGRAPHY

[1] A. Jain, S. P. Ong, G. Hautier, W. Chen, W. D. Richards, H. Dacek, S. Cholia, D. Gunter, D. Skinner, G. Ceder, and K. A. Persson, “Commentary: The Materials Project: A materials genome approach to accelerating materials innovation,” APL Materials, vol. 1, no. 1, p. 011002, 2013.

[2] K. Choudhary, K. F. Garrity, A. C. E. Reid, B. DeCost, F. Tavazza, et al., “The joint automated repository for various integrated simulations (JARVIS) for data-driven materials design,” npj Computational Materials, vol. 6, art. 173, 2020.

[3] S. Curtarolo, W. Setyawan, G. L. W. Hart, M. Jahnatek, R. V. Chepulskii, R. H. Taylor, S. Wang, J. Xue, K. Yang, O. Levy, et al., “AFLOW: An automatic framework for high-throughput materials discovery,” Computational Materials Science, vol. 58, pp. 218–226, 2012.

[4] C. Draxl and M. Scheffler, “NOMAD: The FAIR concept for big data-driven materials science,” MRS Bulletin, vol. 43, no. 9, pp. 676–682, 2018.

[5] A. Merchant, S. Batzner, S. S. Schoenholz, M. Aykol, G. Cheon, and E. D. Cubuk, “Scaling deep learning for materials discovery,” Nature, vol. 624, pp. 80–85, 2023.

[6] C. K. H. Borg, C. Frey, J. Moh, and T. M. Pollock, “Expanded dataset of mechanical properties and observed phases of multi-principal element alloys,” Scientific Data, vol. 7, art. 430, 2020.

[7] Alexandria Materials Database, Interdisciplinary Centre for Advanced Materials Simulation (ICAMS), Ruhr University Bochum. Available: https://alexandria.icams.rub.de/

[8] National Institute of Standards and Technology (NIST), “JARVIS Leaderboard: Benchmark Evaluations for Materials Property Predictions.” Available: https://pages.nist.gov/jarvis_leaderboard/

[9] NASA Ames Research Center, “Thermal Protection Materials Database (TPSX),” NASA Technical Data Systems. Available: https://tpsx.arc.nasa.gov/

[10] NASA Goddard Space Flight Center, “Outgassing Data for Selecting Spacecraft Materials,” NASA Reference Publication 1124. Available: https://outgassing.nasa.gov/

[11] MaterialRegistry, “Standard Engineering Material Registry & Physical Properties API.” Available: https://www.materialregistry.com/docs/api

[12] Outokumpu Stainless AB, “Product Datasheets: Core, Supra, and Forta Duplex Stainless Steels Technical Reference Guides,” 2024.

[13] M. F. Ashby, Materials Selection in Mechanical Design, 5th ed. Oxford, UK: Butterworth-Heinemann, 2016.

[14] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA: Pearson, 2016.

[15] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY: McGraw-Hill, 2019.

[16] Oracle Corporation, “MySQL 8.4 Reference Manual: InnoDB Storage Engine and Optimization Guidelines,” Oracle Technical Documentation, 2024. Available: https://dev.mysql.com/doc/refman/8.4/en/

[17] PingCAP Inc., “TiDB Cloud Serverless Architecture and Distributed MySQL Compatibility Documentation,” 2024. Available: https://docs.pingcap.com/tidbcloud/

[18] OpenJS Foundation, “Express.js 5.0 RESTful Routing and Web Application Framework Documentation,” 2024. Available: https://expressjs.com/

[19] Meta Platforms Inc., “React 18: Concurrent Rendering and Component State Architecture,” 2024. Available: https://react.dev/

[20] S. Poolakkal, B. EB, M. Antony, and S. Xavier, “Starbase: Relational Database System for Engineering Material Selection Source Code Repository,” 2026. Available: https://github.com/sudheeshpoolakkal/starbase